

ADR-01 — Implementación de Service Layer en el Backend

Estado

Propuesto

Contexto

Durante el análisis del sistema **ERP Iglesias** se identificó que los controladores REST contienen parte de la lógica de negocio de la aplicación.

El proyecto ya utiliza **Spring Boot**, **Spring Data JPA** y el **Repository Pattern**, lo cual permite gestionar el acceso a datos de forma adecuada.

Sin embargo, actualmente los controladores realizan múltiples responsabilidades:

- manejo de endpoints HTTP
- aplicación de reglas de negocio
- consulta y persistencia mediante repositorios
- modificación de entidades relacionadas

Esto genera **alto acoplamiento entre la capa web y la lógica de negocio**, lo que dificulta la mantenibilidad del sistema a medida que crece.

Por esta razón se propone introducir una **capa Service** que centralice la lógica de negocio.

Problema

Actualmente algunos controladores contienen lógica de negocio además del manejo de las peticiones HTTP.

Por ejemplo, en `PaymentController` se realizan operaciones como:

- buscar pagos
- cambiar estado de pagos
- actualizar inscripciones
- actualizar ofrendas
- manejar intentos de pago

Ejemplo del código actual:

```
Payment payment = paymentRepository.findById(id)
    .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Pago no
    encontrado"));
```

```
payment.setStatus(PaymentStatus.CONFIRMADO);
paymentRepository.save(payment);

if (payment.getType() == PaymentType.INScripcion_CURSO) {
    Enrollment enrollment =
    enrollmentRepository.findById(payment.getReferenceId())
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
        "Inscripción no encontrada"));
    enrollment.setStatus(EnrollmentStatus.PAGADA);
    enrollmentRepository.save(enrollment);
}
```

Esto provoca:

- violación del **Single Responsibility Principle**
- controladores demasiado grandes
- dificultad para reutilizar lógica de negocio
- dificultad para realizar pruebas unitarias

Decisión Arquitectónica

Se propone introducir una **Service Layer** que centralice la lógica de negocio del sistema.

Los controladores delegarán las operaciones a clases de servicio especializadas.

La arquitectura quedará organizada de la siguiente forma:

```
Controller
↓
Service
↓
Repository
↓
Database
```

El **Repository Pattern** existente **no cambia**, únicamente se agrega la capa Service entre los controladores y los repositorios.

Patrón de Diseño Aplicado

Service Layer Pattern

Este patrón introduce una capa intermedia encargada de encapsular la lógica de negocio del sistema.

Principio SOLID Aplicado

Single Responsibility Principle (SRP)

Cada componente tendrá una única responsabilidad:

Controller → manejar endpoints HTTP Service → lógica de negocio Repository → acceso a datos

Cambio Concreto Propuesto

Se propone crear clases de servicio para gestionar la lógica de negocio de las entidades principales del sistema.

Ejemplo:

```
ChurchService
PersonService
CourseService
EnrollmentService
PaymentService
```

Los controladores dejarán de interactuar directamente con los repositorios para delegar las operaciones en los servicios.

Código Actual (ANTES)

Ejemplo simplificado del controlador actual:

```
@PostMapping("/{id}/confirm")
public PaymentResponse confirm(@PathVariable Long id) {

    Payment payment = paymentRepository.findById(id)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Pago
no encontrado"));

    payment.setStatus(PaymentStatus.CONFIRMADO);
    paymentRepository.save(payment);

    if (payment.getType() == PaymentType.INSCRIPCION_CURSO) {
        Enrollment enrollment =
enrollmentRepository.findById(payment.getReferenceId())
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
"Inscripción no encontrada"));
        enrollment.setStatus(EnrollmentStatus.PAGADA);
        enrollmentRepository.save(enrollment);
    }
}
```

```
    }  
  
    return PaymentResponse.from(payment);  
}
```

Arquitectura actual:

```
Controller  
↓  
Repository  
↓  
Database
```

Mejora Propuesta (DESPUÉS)

Se introduce una clase `PaymentService` que centraliza la lógica de negocio.

PaymentService

```
@Service  
public class PaymentService {  
  
    private final PaymentRepository paymentRepository;  
    private final EnrollmentRepository enrollmentRepository;  
    private final OfferingRepository offeringRepository;  
  
    public PaymentService(  
        PaymentRepository paymentRepository,  
        EnrollmentRepository enrollmentRepository,  
        OfferingRepository offeringRepository) {  
  
        this.paymentRepository = paymentRepository;  
        this.enrollmentRepository = enrollmentRepository;  
        this.offeringRepository = offeringRepository;  
    }  
  
    @Transactional  
    public Payment confirmPayment(Long id) {  
  
        Payment payment = paymentRepository.findById(id)  
            .orElseThrow(() -> new RuntimeException("Pago no encontrado"));  
  
        payment.setStatus(PaymentStatus.CONFIRMADO);  
        paymentRepository.save(payment);  
  
        if (payment.getType() == PaymentType.INScripcion_CURSO) {
```

```
        Enrollment enrollment =
        enrollmentRepository.findById(payment.getReferenceId())
            .orElseThrow(() -> new RuntimeException("Inscripción no
        encontrada"));

        enrollment.setStatus(EnrollmentStatus.PAGADA);
        enrollmentRepository.save(enrollment);

    } else if (payment.getType() == PaymentType.OFRENDAS) {

        Offering offering =
        offeringRepository.findById(payment.getReferenceId())
            .orElseThrow(() -> new RuntimeException("Ofrenda no encontrada"));

        offering.setStatus(OfferingStatus.REGISTRADA);
        offeringRepository.save(offering);
    }

    return payment;
}
```

Controller después del cambio

```
@PostMapping("/{id}/confirm")
public PaymentResponse confirm(@PathVariable Long id) {

    Payment payment = paymentService.confirmPayment(id);

    return PaymentResponse.from(payment);
}
```

Arquitectura resultante:

```
Controller
↓
Service
↓
Repository
↓
Database
```

Beneficios Arquitectónicos

La implementación de Service Layer proporciona:

- mejor separación de responsabilidades
- reducción del acoplamiento entre capas
- mayor mantenibilidad del sistema
- mejor organización del código
- mayor facilidad para pruebas unitarias
- soporte para operaciones transaccionales mediante `@Transactional`

Impacto en el Sistema

El cambio no afecta el comportamiento funcional del sistema.

Los endpoints REST permanecen iguales, pero ahora delegan la lógica de negocio a la capa **Service**.

Resultado Arquitectónico

Arquitectura final propuesta:

```
Controller
↓
Service
↓
Repository
↓
Database
```

Esta estructura mejora la organización del sistema, facilita su evolución y permite aplicar de forma adecuada los principios de diseño y arquitectura de software.